



Objektno orijentirano programiranje

Ljetni ispitni rok

6.7.2021.

Ispit nosi ukupno 50 bodova i piše se 150 minuta.

U zadacima nije potrebno pisati dio u kojem se uključuju klase ili paketi klasa (import)

Rješenja je potrebno pisati na stranice ispita. Ako na nekoj stranici ne budete imali dovoljno mjesta, ostatak možete napisati na zadnju (praznu) stranicu ispita te isto naznačiti na zadatku kojeg nadopunjujete.

IZJAVA

Tijekom ove provjere znanja neću od drugoga primiti niti drugome pružiti pomoć te se neću koristiti nedopuštenim sredstvima.

Ove su radnje povreda Kodeksa ponašanja te mogu uzrokovati trajno isključenje s Fakulteta.

Zdravstveno stanje dozvoljava mi pisanje ovog ispita.

Vlastoručni potpis studenta: _____

1. zadatak (10 bodova)

Potrebno je modelirati sustav parkirališta s jednim ulazom i jednim izlazom. **Vaš zadatak je nadopuniti predložak UML dijagrama klasa ovog sustava koji se nalazi na sljedećoj stranici.** Pritom **možete dodati nove klase** (ili sučelja) ako smatrate da je to potrebno. **Nije potrebno pisati programski kod sustava i to neće donositi nikakve bodove.** Na ovom predlošku pažljivo navedite sljedeće:

- oznake za sučelje (**I**), enumeraciju (**E**), apstraktnu (**A**) ili običnu klasu (**C**)
- oznake za apstraktnu metodu (**A**) te oznake za statičku (**S**) i finalnu (**F**) metodu ili atribut
- modifikatore vidljivosti metoda i atributa (+ public, # protected, - private, ~ package-private)
- tipove povratnih vrijednosti i argumenata za metode te tipove atributa
- odgovarajuće strelice da naznačite nasljeđivanje klasa (puna linija $\longrightarrow$) ili implementaciju sučelja (iscrtkana linija $-\cdots\longrightarrow$)

Parkiralište `ParkingLot` ima zabilježen skup parkirnih mjesta `ParkingSpot` (tipa `HashSet`) te bilježi svaku izdanu parkirnu kartu `Ticket`. Karte su posložene u listu (tipa `ArrayList`). Parkiralište također ima metode za: dodavanje parkirne karte `Ticket` u listu (`addTicket`), dodavanje novih parkirnih mjesta `ParkingSpot` u skup (`addSpot`) te metodu za dohvaćanje karte iz liste (`getTicket`). Osim njih, parkiralište mora moći primiti vozilo metodom `enterLot`, pri čemu se izdaje (instancira) parkirna karta (`Ticket`) te otpustiti vozilo metodom `exitLot`. Pri izlasku se predaje i provjerava parkirna karta (`Ticket`).

Dvije parkirne karte (tipa `Ticket`) se u listi međusobno razlikuju po svojem cjelobrojnom identifikatoru `id`. Karte osim toga imaju podatak o tome jesu li plaćene te vrijeme ulaska tipa `LocalDateTime`. Klasu `LocalDateTime` nije potrebno navoditi u dijagramu. Karta također ima svoj konstruktor koji prima i zabilježava parkirno mjesto `ParkingSpot` koje je vozilo zauzelo. Osim toga, karti se mora moći postaviti i provjeriti stanje je li plaćena (`setPaid` i `isPaid`), i dohvatiti identifikator (`getID`).

Parkirno mjesto `ParkingSpot` sadrži podatak o tome je li zauzeto (`occupied`), cjelobrojni identifikator po kojem se dva mjesta međusobno razlikuju (`id`) i svoju poziciju `String`. Parkirno mjesto mora imati metode za provjeru dostupnosti mjesta, zauzeće i oslobađanje te dohvat pozicije i identifikatora. Identifikator i pozicija se postavljaju konstruktorom. Parkirno mjesto može biti standardno (`StandardSpot`), za invalide (`HandicappedSpot`) ili pak za punjenje električnog automobila (`ChargingSpot`). Mjesto za invalide sadrži informaciju o tome je li dodatno široko (`wideSpot`). Mjesto za punjenje električnog automobila sadrži IP adresu (tipa `String`) za komunikaciju s punjačem, a sadrži i informaciju o tome je podržava li brzo punjenje (`quickcharge`) te informaciju o isporučenoj energiji (`kWhCount`) pri punjenju tipa `double`. Dohvat isporučene energije moguć je metodom `getkWhCount`.

Karta `Ticket` mora se moći platiti kako nalaže `Payable`. `Payable` mora podržavati sve oblike plaćanja, uključujući i one koji još nisu predviđeni u ovom tekstu. Metoda `pay` prima takav općeniti oblik plaćanja koji se izvršava ovisno o primljenoj klasi. `Payable` osim toga ima i metodu za izračun cijene.

Svako plaćanje parkirne karte `Payment` ima člansku varijablu (`amount`) koja sadrži iznos plaćanja te cjelobrojni identifikator plaćanja `id`. Plaćanje se može izvršiti gotovinom (`CashPayment`) i karticom (`CardPayment`) i obavlja se prije izlaska s parkirališta. Za plaćanje gotovinom mora se moći izračunati povrat novca s `calculateChange` te se bilježi primljeni iznos novca `receivedAmount`. Za plaćanje karticama potrebne su metode `connectPOS` i `makeTransaction` koje se koriste za spajanje i izvršavanje transakcije.

--

Payable

Ticket

ParkingLot

Payment

ParkingSpot

CashPayment

CardPayment

StandardSpot

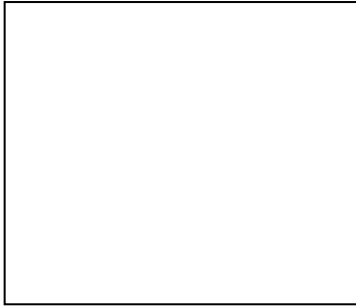
ChargingSpot

HandicappedSpot

mogu se koristiti po potrebi.

Klasa smoker ima javne atribute koji predstavljaju komparatore po određenim komponentama. BY_TOTAL_SHELF_SPACE je komparator koji dva smokera uspoređuje po **ukupnoj** zapremini **svih** polica. BY_MIN je komparator koji smokere uspoređuje po minimalnoj temperaturi, dok je BY_MIN_THAN_MAX komparator koji smokere uspoređuje prvo po minimalnoj a potom po maksimalnoj temperaturi. Kod implementacije zadnje spomenutog komparatora obavezno je korištenje komparatora BY_MIN.

```
public class Smoker {  
  
    private String name;  
    private String producer;  
    private Type type;  
    private Set<Long> shelfSpace;  
    private int minTemp;  
    private int maxTemp;  
  
    enum  
  
        public static final Comparator<Smoker> BY_TOTAL_SHELF_SPACE = new  
ByTotalShelfSpaceComparator();  
  
        public static final Comparator<Smoker> BY_MIN =  
  
  
        public static final Comparator<Smoker> BY_MIN_THAN_MAX =  
  
  
    private static class ByTotalShelfSpaceComparator {  
  
  
  
  
  
  
  
  
  
}
```



```
@Override  
public int hashCode() {
```

```
}
```

```
@Override  
public boolean equals(Object obj) {
```

```
}
```

```
@Override  
public int compareTo(Smoker s) {
```

```
}
```

```
//getteri, setteri i konstruktor su već implementirani  
}
```

3. zadatak (10 bodova)

U zadanom sustavu registrirani korisnici su modelirani klasom User koja sadrži sljedeće atribute: `userId:Integer`, `username:String`, `password:String`, `email:String`, `hoursActive:Integer`, `transactionsCount:Integer`, `currentSaldo:Integer`, te odgovarajuće gettere i settere i konstruktor koji postavlja sve atribute (u danom redoslijedu).

Kako bi se sustav mogao pokrenuti na novom stroju potrebno je najprije implementirati metodu:

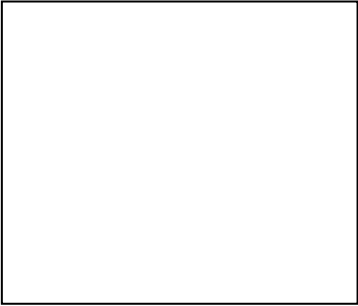
- `private void importData(String filename)` - metoda kao argument prima putanju do datoteke u kojoj se nalaze podaci o svim postojećim korisnicima u sljedećem obliku (svaki redak sadrži jedan zapis):
`userId;username;password;email;hoursActive;transactionsCount;currentSaldo`
Podatke je potrebno učitati u mapu `users<Integer, User>` u kojoj se nalaze podaci o svim korisnicima sustava (ključ je id korisnika, a vrijednost odgovarajući objekt), a pritom je potrebno uzeti u obzir mogućnost da neke linije nisu napisane u odgovarajućem formatu, te ih je onda potrebno zanemariti i ispisati na standardni izlaz poruku: "ERR";
*ako u mapi postoji korisnik koji ima jednak `userId` kao i korisnik koji se želi dodati, potrebno je preskočiti takav unos, odnosno ostaviti postojeći zapis u mapi;

Zatim je potrebno implementirati sljedeće metode koristeći koleksijske tokove (Stream API) kako bi sustav omogućio nove funkcionalnosti:

- `private void notifyNonActiveUsers()` - svim korisnicima koji su sustav koristili manje od 3 sata potrebno je poslati notifikaciju elektroničkom poštom. Slanje emaila je potrebno ostvariti pozivom gotove metode `sendNotification(List<String> usersToNotify)`, kojoj je kao argument potrebno predati listu email adresa korisnika koje je potrebno kontaktirati;
- `private Map<Integer, Double> convertMoneyToAnotherCurrency(double rate)` - metoda treba iz postojeće mape `users` generirati izlaznu mapu u kojoj je ključ također id korisnika, a pripadajuća vrijednost je korisnikov saldo pomnožen tečajem koji se metodi predaje kao parametar;
- `private void mergeNewUsers(Map<Integer,User> newUsers)` - metoda treba u postojeću mapu `users` dodati sve korisnike iz mape `newUsers` koja se predaje kao parametar ovoj metodi. Ukoliko u mapi `users` postoji korisnik koji ima jednak `userId` kao i neki korisnik iz mape `newUsers`, potrebno je ostaviti postojeći zapis korisnika u mapi `users`, ali je njegov saldo potrebno uvećati za vrijednost salda istog korisnika iz mape `newUsers`. Funkcionalnost je potrebno izvesti koristeći metodu `merge` čiji je JavaDoc prikazan na slici ispod.

```
default V merge(K key, V value, BiFunction<? super V,? super V,? extends V> remappingFunction)
```

If the specified key is not already associated with a value or is associated with null, associates it with the given non-null value. Otherwise, replaces the associated value with the results of the given remapping function, or removes if the result is null.



```
public class Main {  
    private Map<Integer, User> users = new HashMap<>();  
    private void importData(String filename) throws  
        IOException {
```

```
}
```

```
private void notifyNonActiveUsers() {  
    sendNotification(users.
```

```
);  
}
```

```
private Map<Integer,Double> convertMoneyToAnotherCurrency(double rate) {  
    return users.
```

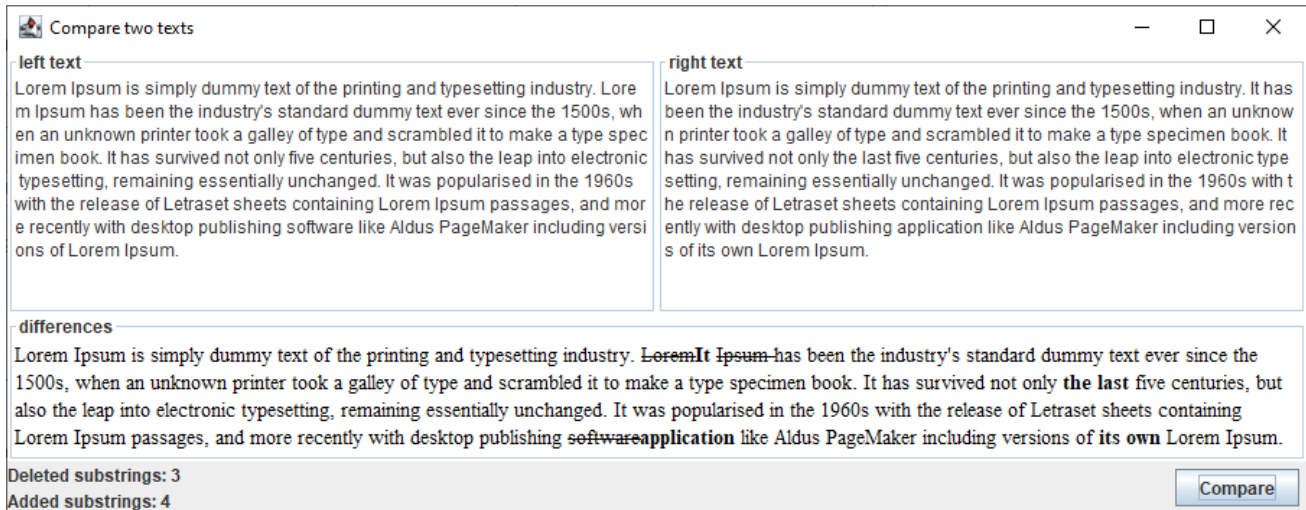
```
}
```

```
private void mergeNewUsers(Map<Integer,User> newUsers) {
```

```
    }  
}
```

4. zadatak (10 bodova)

Vaš zadatak je nadopuniti programski kod aplikacije s grafičkim korisničkim sučeljem koja je prikazana ispod. Klikom na gumb *Compare*, aplikacija treba prikazati razlike između tekstova te ispisati broj uklonjenih (*deleted*) i dodanih (*added*) podstringova. **U ovom zadatku trebate stvoriti i razmjestiti grafičke komponente, definirati slušača klika na gumb te dovršiti metodu main.** Posao pronalaženja razlika (tekstova) je zahtjevan te ga treba obaviti korištenjem vanjske klase *DiffWorker* koja nasljeđuje klasu *StringWorker*, a trebate ju implementirati u sljedećem zadatku (5. zadatak). U ovom zadatku stoga **ne trebate implementirati klasu *DiffWorker*, već stvoriti i pokrenuti njenu instancu prilikom klika na gumb.**



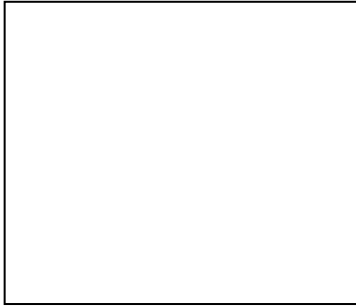
```
public final class DiffFrame extends JFrame {

    private JTextArea textAreaLeft, textAreaRight;
    private JTextPane textPane;
    private JLabel deletedLabel, addedLabel; //counter labels

    public DiffFrame() {

        //create, set up and add a text pane
        textPane = new JTextPane();
        textPane.setContentType("text/html");
        textPane.setEditable(false);
        textPane.setPreferredSize(new Dimension(0, 200));
        textPane.setBorder(BorderFactory.createTitledBorder("differences"));
        add(textPane);

        //create and set up left and right text areas
        textAreaLeft = new JTextArea(10, 40);
        textAreaRight = new JTextArea(10, 40);
        textAreaLeft.setLineWrap(true);
        textAreaRight.setLineWrap(true);
        textAreaLeft.setBorder(BorderFactory.createTitledBorder("left text"));
        textAreaRight.setBorder(BorderFactory.createTitledBorder("right text"));
```

```
//TODO - add the left and right text area
```

```
//TODO - add statistic counters
```

```
//TODO - add the button
```

```
//TODO - define action on button click
```

```
} //end of DiffFrame constructor
```

```
public static void main(String[] args) {  
    SwingUtilities.invokeLater(() -> {  
        //TODO - create and show the frame
```

```
    });  
} //end of main  
} //end of DiffFrame
```

5. zadatak (10 bodova)

Vaš zadatak je nadopuniti programski kod klase `DiffWorker`. U ovoj klasi trebate nadopuniti programski kod metode `doInBackground` te implementirati metode `process`, `done` i `countOccurrences`. U metodi `doInBackground` je već implementiran dio programskog koda u kojem ona dohvaća dva teksta (koja treba usporediti), dijeli ih na rečenice korištenjem **gotove** metode `splitSentences`, uspoređuje rečenicu po rečenicu korištenjem vanjske klase `DiffRowGenerator`, generira razliku svake rečenice u obliku HTML koda te u petlji `for` pretvara HTML kod svake razlike u njezinu reprezentaciju `rowString` tipa

`String` (koju treba nadodati u povratnu vrijednost) i stvara međurezultat tipa `Pair<Integer>` koji predstavlja par vrijednosti tipa `Integer`, gdje prva i druga vrijednost predstavljaju broj obrisanih tj. dodanih riječi u rečenici. **Ne trebate implementirati klasu `Pair<T>`** koja ima konstruktor za postavljanje vrijednosti para i metode `getFirst` i `getSecond` koje su *getteri* za dohvat prve i druge vrijednosti u paru. **U navedenu petlju `for` dodajte programski kod za objavu međurezultata i ažuriranje povratne vrijednosti ove metode.** Osim toga **implementirajte metode `process` i `done` na način da prva ažurira prikaz statističkih brojača obrađujući međurezultate, a druga postavlja tekst grafičke komponente tipa `JTextPane` na povratnu vrijednost metode `doInBackground`.** Na kraju **implementirajte i metodu `countOccurrences`** koja broji koliko se puta `string` koji je predan kao drugi argument (`find`) pojavljuje u `stringu` koji je predan kao prvi argument (`string`).

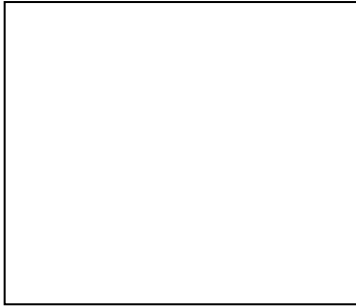
```
public class DiffWorker extends SwingWorker<String, Pair<Integer>> {
    private int deleted = 0, added = 0; //counters
    private String leftText, rightText;
    private JLabel deletedLabel, addedLabel; //counter labels
    private JTextPane textPane;

    public DiffWorker(String leftText, String rightText, JLabel deletedLabel, JLabel
addedLabel, JTextPane textPane) {
        this.leftText = leftText; this.rightText = rightText; this.deletedLabel =
deletedLabel; this.addedLabel = addedLabel; this.textPane = textPane;
    } //end of Diffworker constructor

    private List<String> splitSentences(String text) {
        List<String> result = new LinkedList<>();
        BreakIterator breakIterator = BreakIterator.getSentenceInstance();
        breakIterator.setText(text);
        int index = 0;
        while (breakIterator.next() != BreakIterator.DONE) {
            String sentence = text.substring(index, breakIterator.current());
            result.add(sentence);
            index = breakIterator.current();
        }
        return result;
    } //end of splitSentences

    @Override
    protected String doInBackground() throws Exception {
        //get differences
        DiffRowGenerator generator = DiffRowGenerator.create()
            .showInlineDiffs(true).mergeOriginalRevised(true)
            .inlineDiffByWord(true)
            .oldTag((t, f) -> f ? "<strike>" : "</strike>") //tag for deleted words
            .newTag((t, f) -> f ? "<b>" : "</b>") //tag for added words
            .build();

        List<DiffRow> rows = generator.generateDiffRows(
            splitSentences(leftText), splitSentences(rightText));
```



```
StringBuilder sb = new StringBuilder("<html>");

//do something with each DiffRow in rows
for (DiffRow row : rows) {
    String rowString = row.getOldLine();
    Pair<Integer> pair = new Pair<>(
        countOccurrences(rowString, "<strike>"),
        countOccurrences(rowString, "<b>"));

    //TODO - update return value, publish chunks

}
sb.append("</html>");
return sb.toString();
} //end of doInBackground method

@Override
protected void process(List<Pair<Integer>> chunks) {
    //TODO

} //end of process method

@Override
protected void done() {
    //TODO

} //end of done method

private int countOccurrences(String string, String find) {
    //TODO

} //end of countOccurrences method
} //end of DiffWorker class
```



Na ovoj stranici možete napisati kod koji vam nije stao na prethodne stranice.